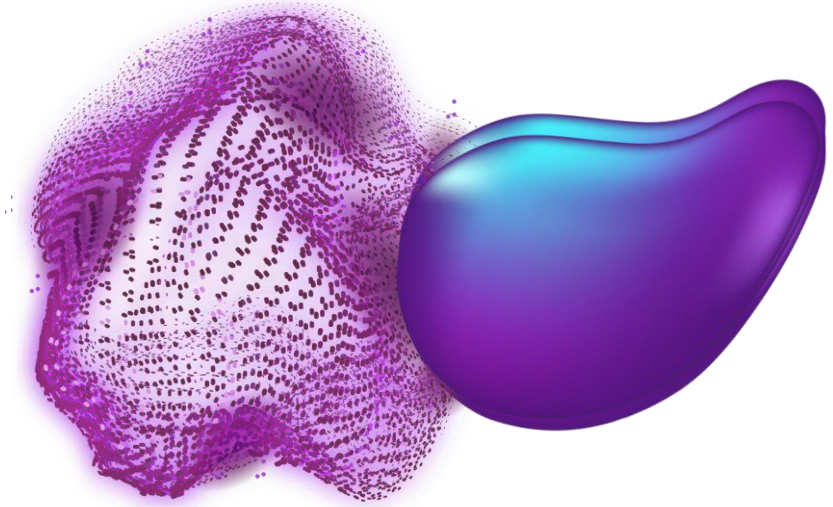


Object Oriented Programming

Lecture 6

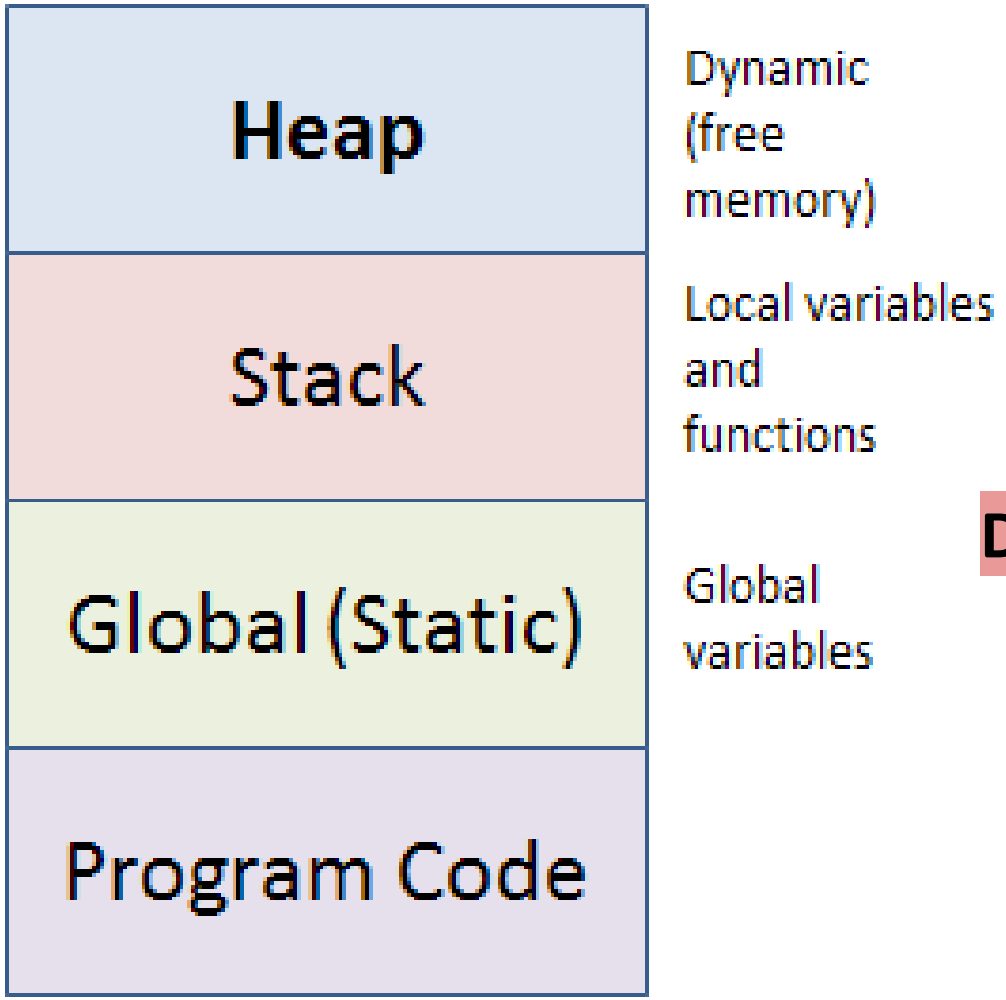
Dynamic Memory Allocation

```
int main(){
    const int SIZE=10000000;
    int arr[SIZE];
```

```
First-chance exception at 0x00F31B97 in Dynamic Memory Allocation.exe: 0xC00000FD: Stack overflow (parameters: 0x00000000, 0x00D02000).
Unhandled exception at 0x00F31B97 in Dynamic Memory Allocation.exe: 0xC00000FD: Stack overflow (parameters: 0x00000000, 0x00D02000).
```

```
int numofstudent;
cout << "Enter total number of students: ";
cin >> numofstudent;
int students[numofstudent];
```

Stack & Heap ?

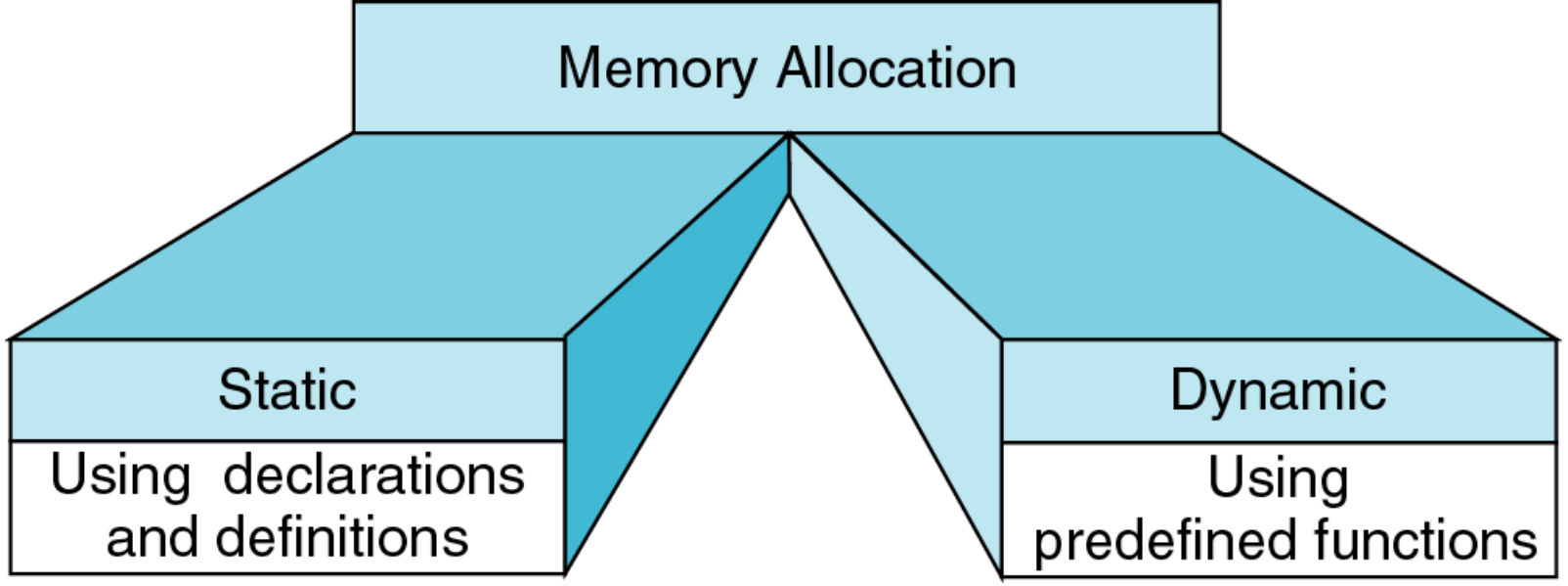


STATIC MEMORY

- Memory is allocated at **compilation** time
- Declaration of large memory variables (arrays) must be **determined** before the program is compiled.
  - ✓ **int students[10];**
  - ✓ What if we don't know the exact number of students in advance?

DYNAMIC MEMORY

- We can "**ask**" for memory while the program is running.
- We can **allocate** and **release** memory ourselves! "**Dynamically**"
- More powerful programming ability (also more potential for errors!)



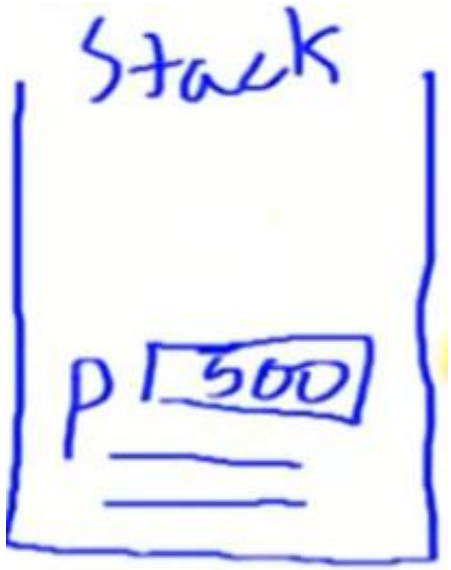

```
{
  int a[200];
  int *ptr = a;
  ...
}
```

- *Memory* is acquired automatically
- *Memory* is returned automatically when object goes out of scope

```
int *ptr = new int[200];
...
delete [] ptr;
```

- *Memory* is acquired by **program** with an allocation request
  - **new** operation
- Dynamic objects can exist beyond the function in which they were allocated
- *Object memory* is returned by a **deallocation** request
  - **delete** operation

```
int main(){
    int *p=new int;
```



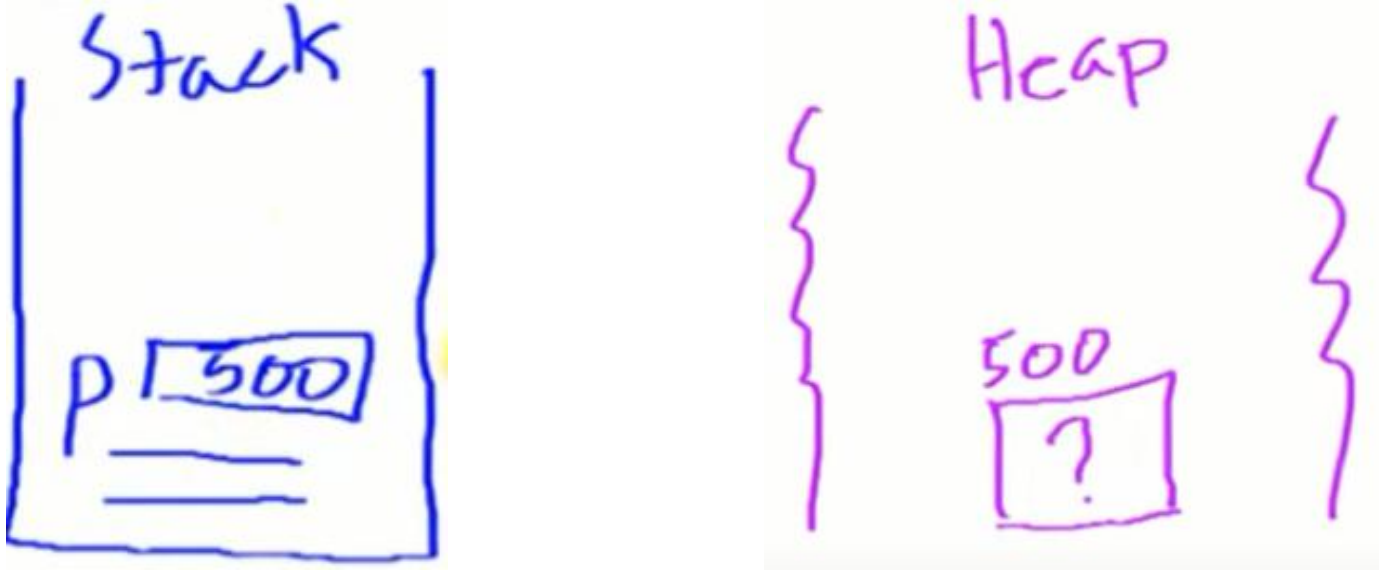
int *p

new int

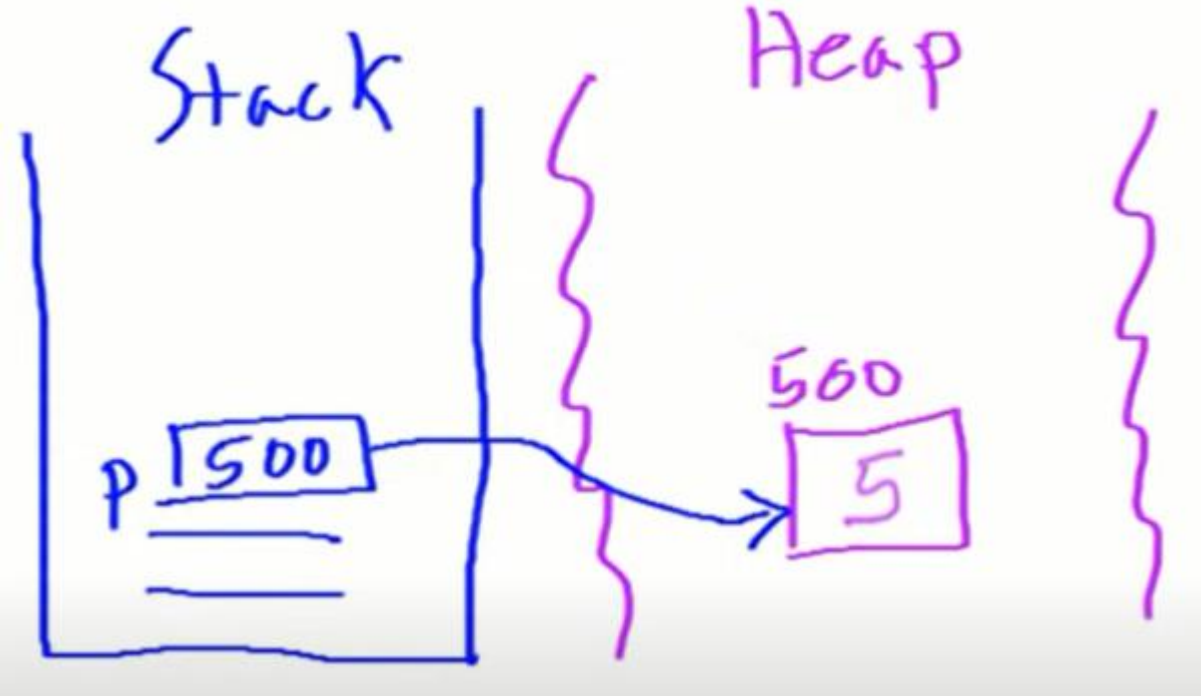
Pointer itself is stored in the stack part

Ask OS to find a segment Of memory (**“unused” memory**) of 4 btyes (for int) , then OS return the **address** of the reserved memory space

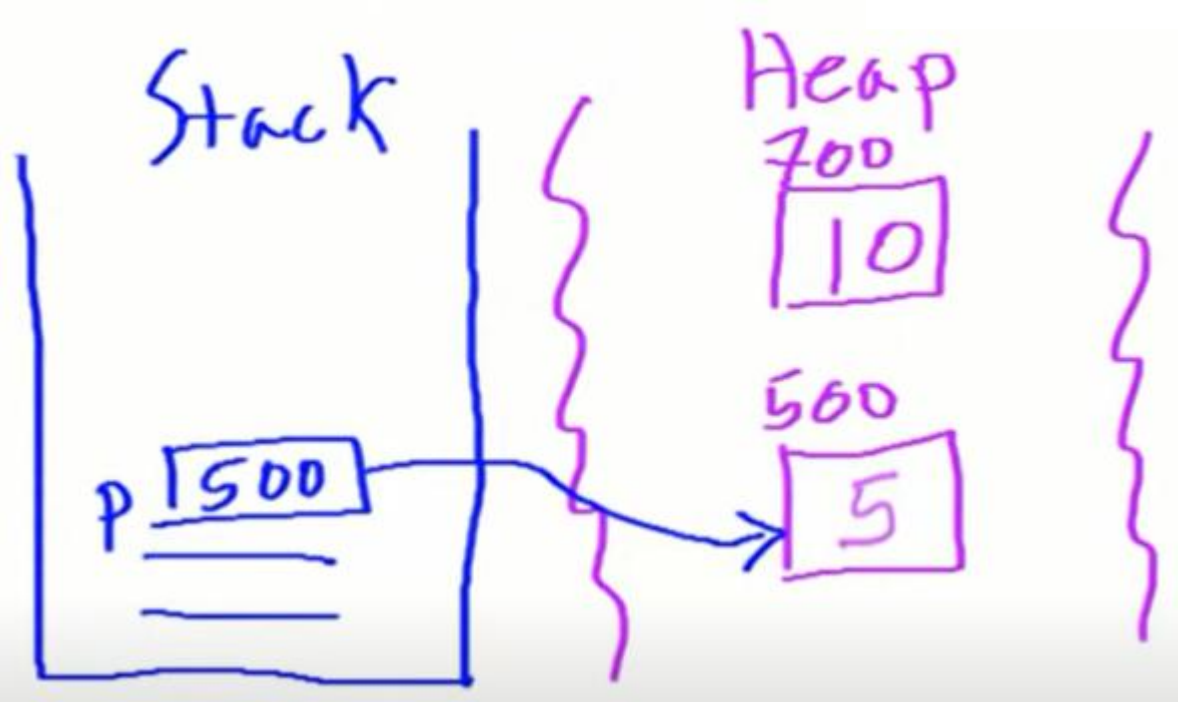
So, you need a pointer to the type you requested.
If you request <int>, you need an <int> pointer.
If you request <float>, you need a <float> pointer.

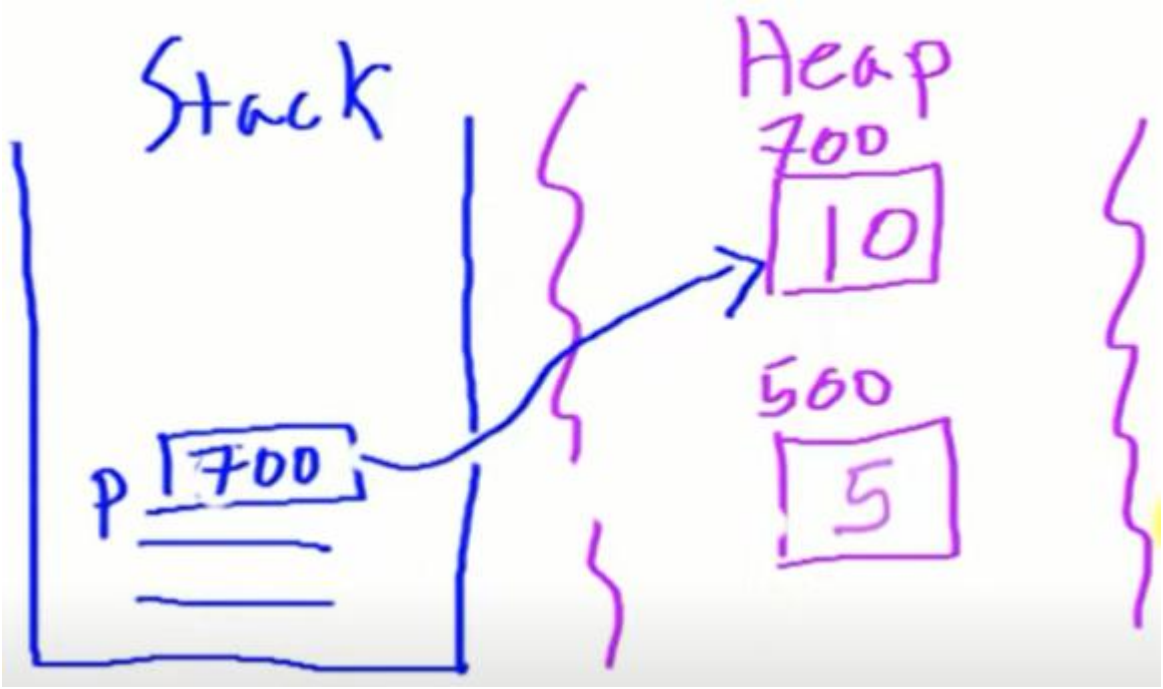



```
int *p=new int;
*p=5;
```



```
int *p=new int;
*p=5;
p=new int(10);
```



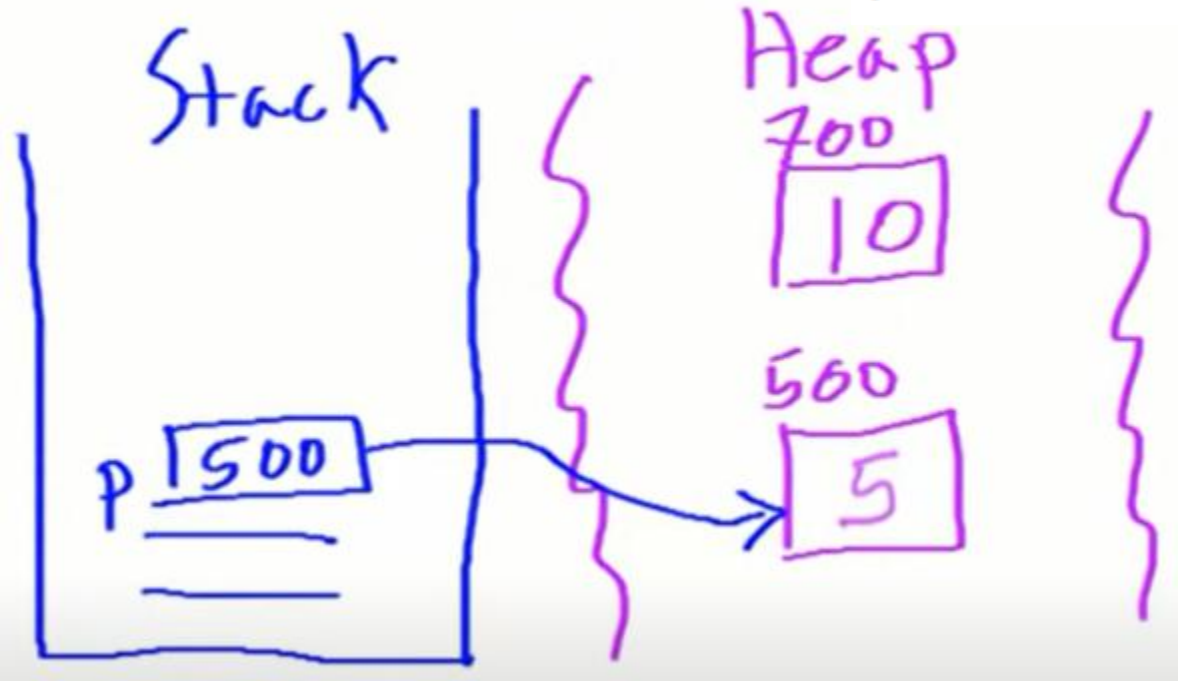


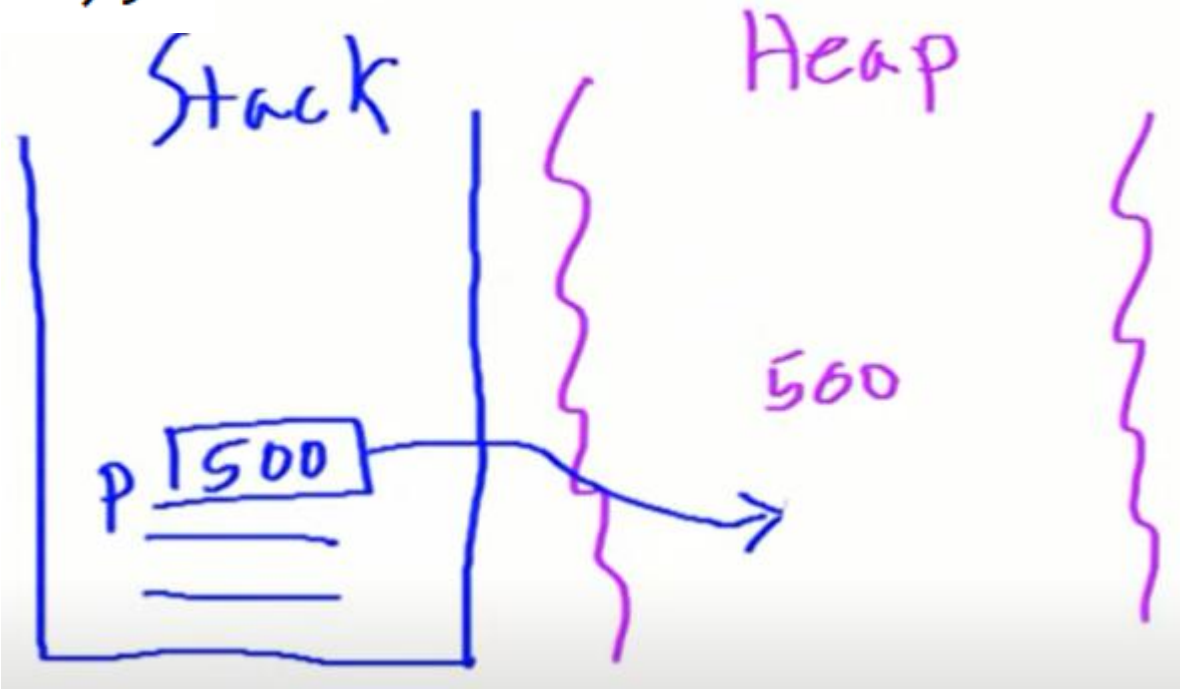
What is the main problem here ?!

Garbage Problem

- Make sure to **delete** memory when finished
- **Memory Leak Problem**

```
int *p=new int;
*p=5;
delete p;
p=new int(10);
```





delete keyword doesn't delete the pointer itself, it delete what the pointer refers to.

Delete → 5

To delete pointer make it point to nothing NULL **p=NULL;**

Create a Dynamic Array

```
// uninitialized Dynamic Array
int *dynarr1=new int[500];   garbage values
cout<<"dynarr1[10]= "<<dynarr1[10]<<endl;
// initialized to Zero
int *dynarr2=new int[500]();   (slower)
cout<<"dynarr2[10]= "<<dynarr2[10]<<endl;
```

```
dynarr1[10]= -842150451
dynarr2[10]= 0
```

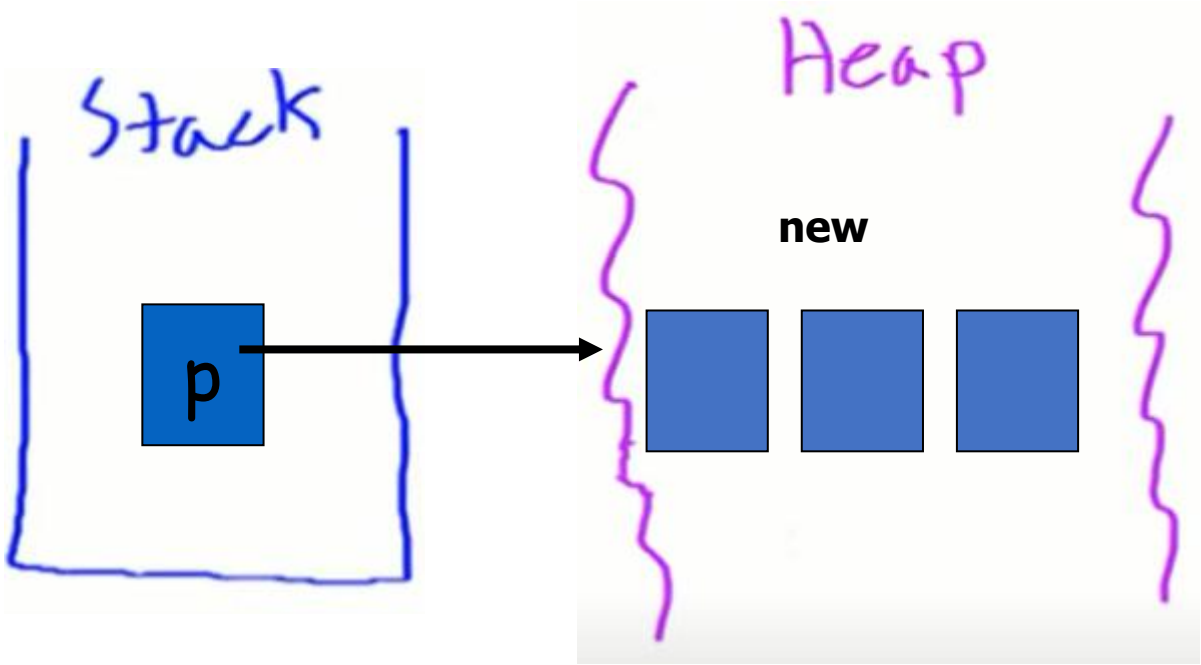
The memory allocated for the array must be **manually released**, or else the program will have a **memory leak**.

- Another array creation syntax that we will not use: **int fixedarr[500];**
- A fixed array; initialized at **declaration**; can **never** be **resized**.
- Stored in a different place in memory; the **stack** and the Dynamic array uses the **heap**.

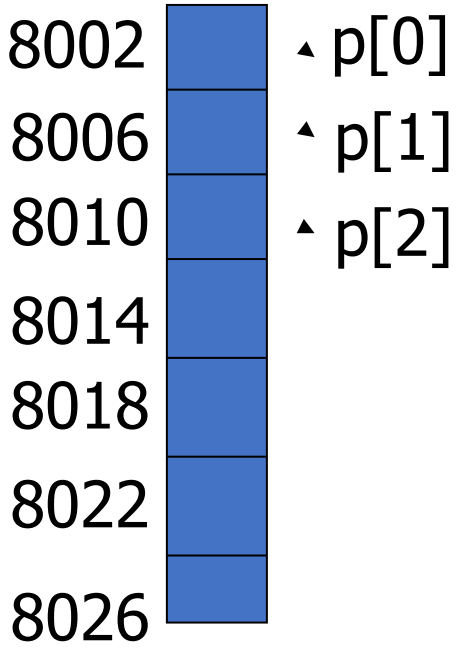
Create a Dynamic Array

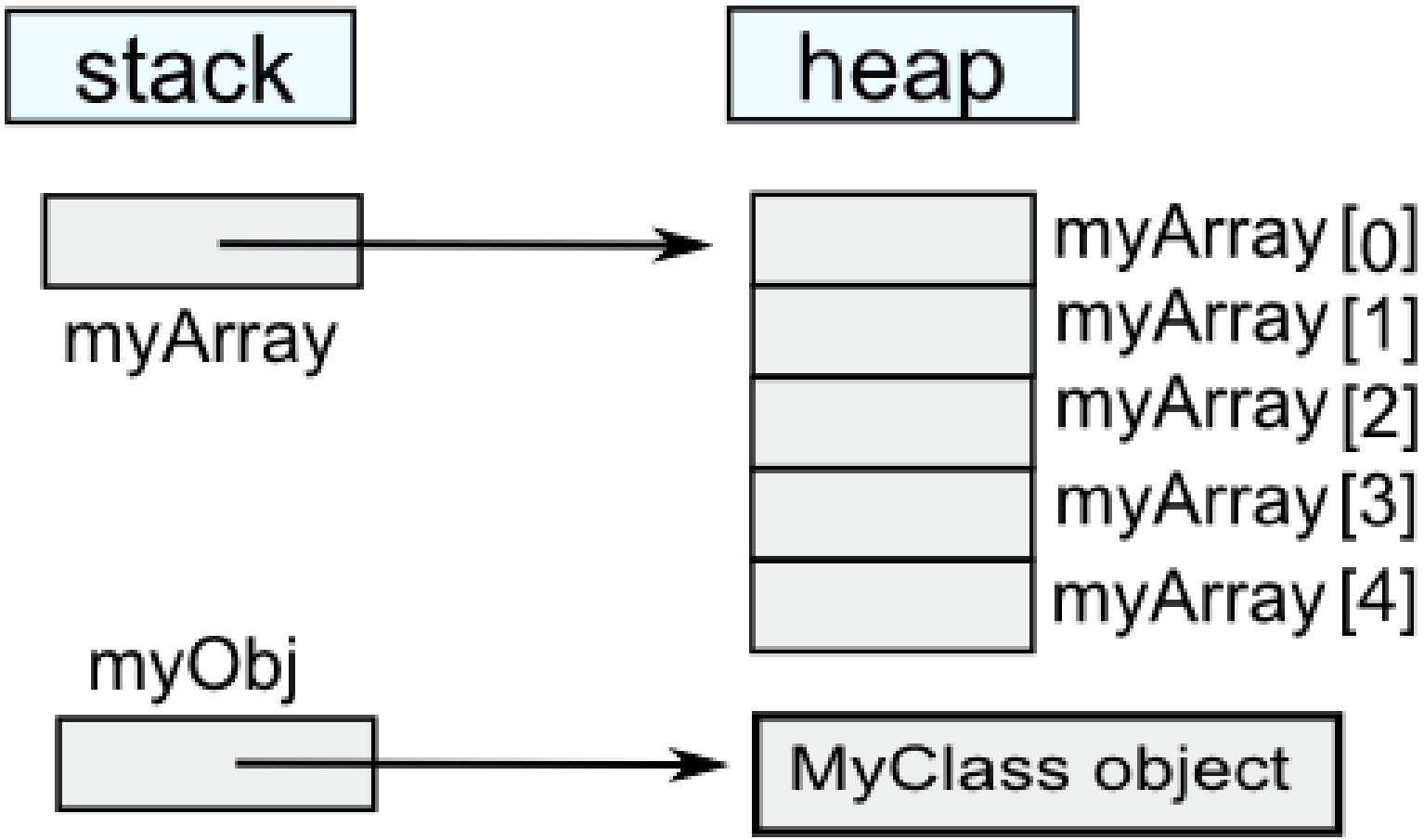
p = new int[n];

new returns the address to the "start" of the memory it allocated




```
int *p;
p = new int[7];
//p is assigned 8002
p[0] = *(p) = *(p+0) = ?
p[1] = *(p+1) = ?
p[2] = *(p+2) = ?
```

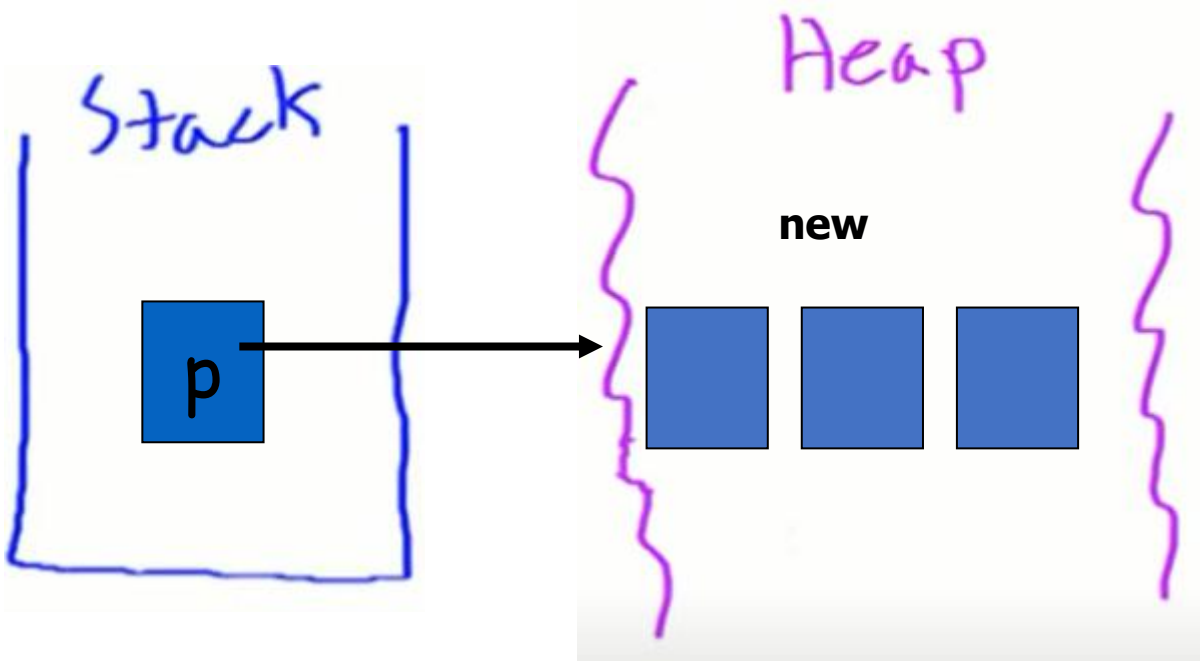





```
double *myArray = new double[5]
MyClass *myObj = new MyClass
```

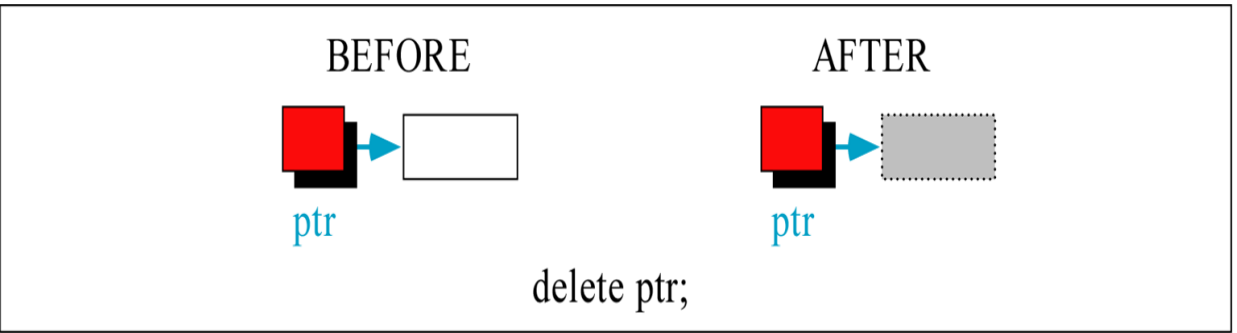
Freeing Memory from array

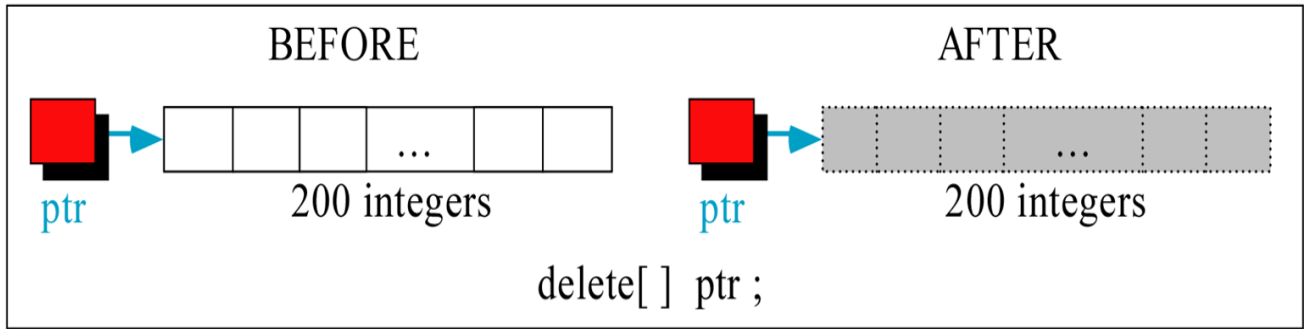
The memory you request by "new" can not be used again until you explicitly release it via "delete".
We call this "**freeing**" or "**releasing**" or "**deleting**" memory.




```
int *dynarr1=new int[500]();
delete []dynarr1;
```

The value of p will not change! However, it is now no longer points to a valid memory address.

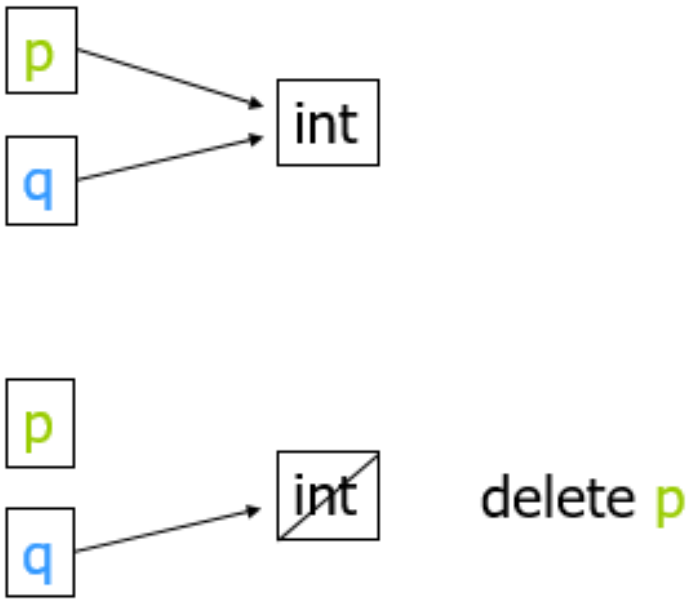


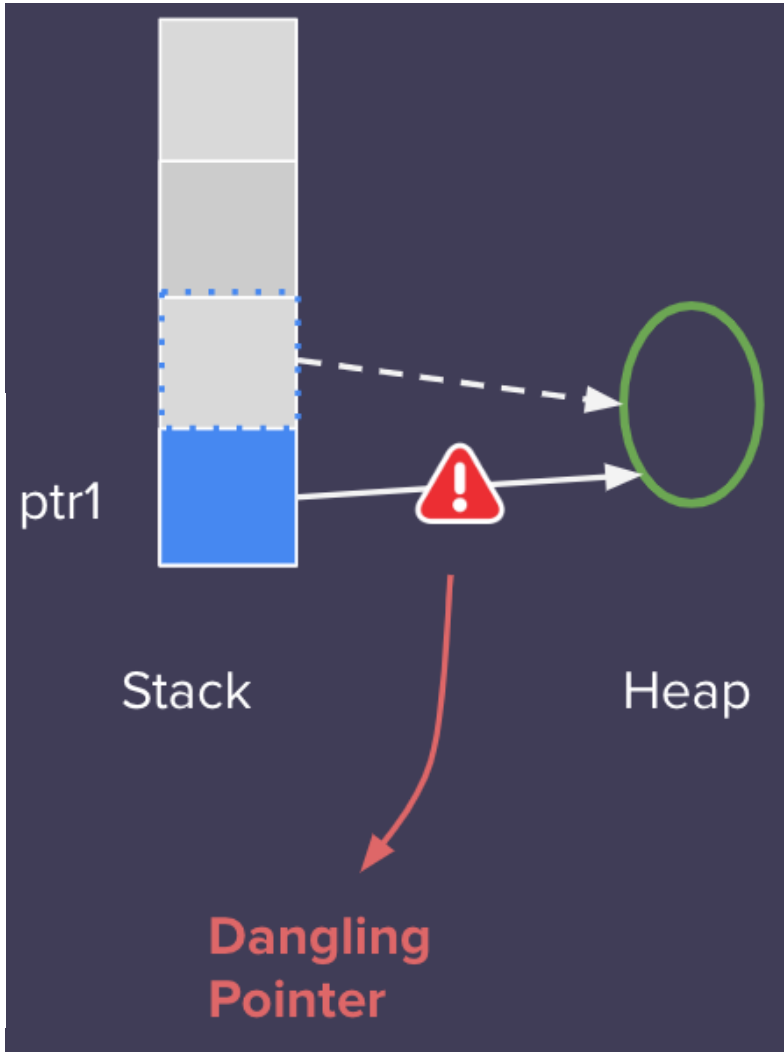


Dangling pointer problem ?

Be careful when you delete memory pointed to by **p**, you are not erasing a location that some other pointer **q** is pointing to.

```
int *p, *q;
p = new int;
q = p;
delete p;
// q is a dangling pointer
```





From Stack to Heap …

```
int main(){
    const int SIZE=10000000;
    int arr[SIZE];
```

```
int main(){
    int *arrptr=new int[10000000];  //heap  //DMA
```

```
    int size;
    cin >> size;
    int *arrptr=new int[size];
```

```
int numofstudent;
cout << "Enter total number of students: ";
cin >> numofstudent;
int students[numofstudent];
```

```
int numofstudent;
cout << "Enter total number of students: ";
cin >> numofstudent;
int *students=new int[numofstudent];
```

Some Notes:

```
int x=5;
int *intptr= &x;    //* isn't a dereference operator
cout<< *intptr <<endl; //* a dereference operator
*intptr=7;
cout<< *intptr <<endl;


int *intptr;
cout<< *intptr <<endl; // Debug problem // Crash
*intptr=7; //debug problem
cout<< *intptr <<endl;  //debug problem

int *arrptr= new int[5];
arrptr[2]=15;
//delete array itself from heap to free some space for other program
delete []arrptr;
//now arrptr pointer point to nothing, so assign it to NULL
arrptr=NULL;
```

Exercise 1

```
int * p1 = new int(5);  //0x12
int * p2 = new int(7);  //0x4
int * p3 = new int(8);  //0x20
*p1 = *p2;
cout << p1 << " " << *p1 << endl;
p1 = p2;
cout << p1 << " " << *p1 << endl;
delete p1;
//what could go here?cause we need that
cout << *p3 <<endl;
```

How many of these lines of code could legally be the next line?

A. 0-2 of them
B. 3 of them
C. 4 of them
D. 5+ of them

```
delete p2;
delete p3; //1
*p1 = 10;
p1 = p3;  //2
cout << p1 << endl; //3
cout << *p2 << endl; //4
```

Exercise 2

```
int x =20;
int* p = new int;
*p = 30;
cout << *p << " " << x << endl;
delete p;
p = &x;
delete p; // Error
cout << *p << " " << x << endl;
```

Exercise 3

```
const int SIZE = 10000000;
int i;
int A[SIZE];  // Stack Overflow Error
for(i=0; i<SIZE; i++)
    A[i] = i;
int *B= new int[SIZE];
for(i=0; i<SIZE; i++)
    B[i] = A[i];
for(i=0; i<SIZE; i++)
    cout << A[i] << " " << B[i] << endl;
```

Exercise 4

```
int *p, *r;
p = new int;
r = new int;
p = 1;  //compile time error *p=1
r = p;
```

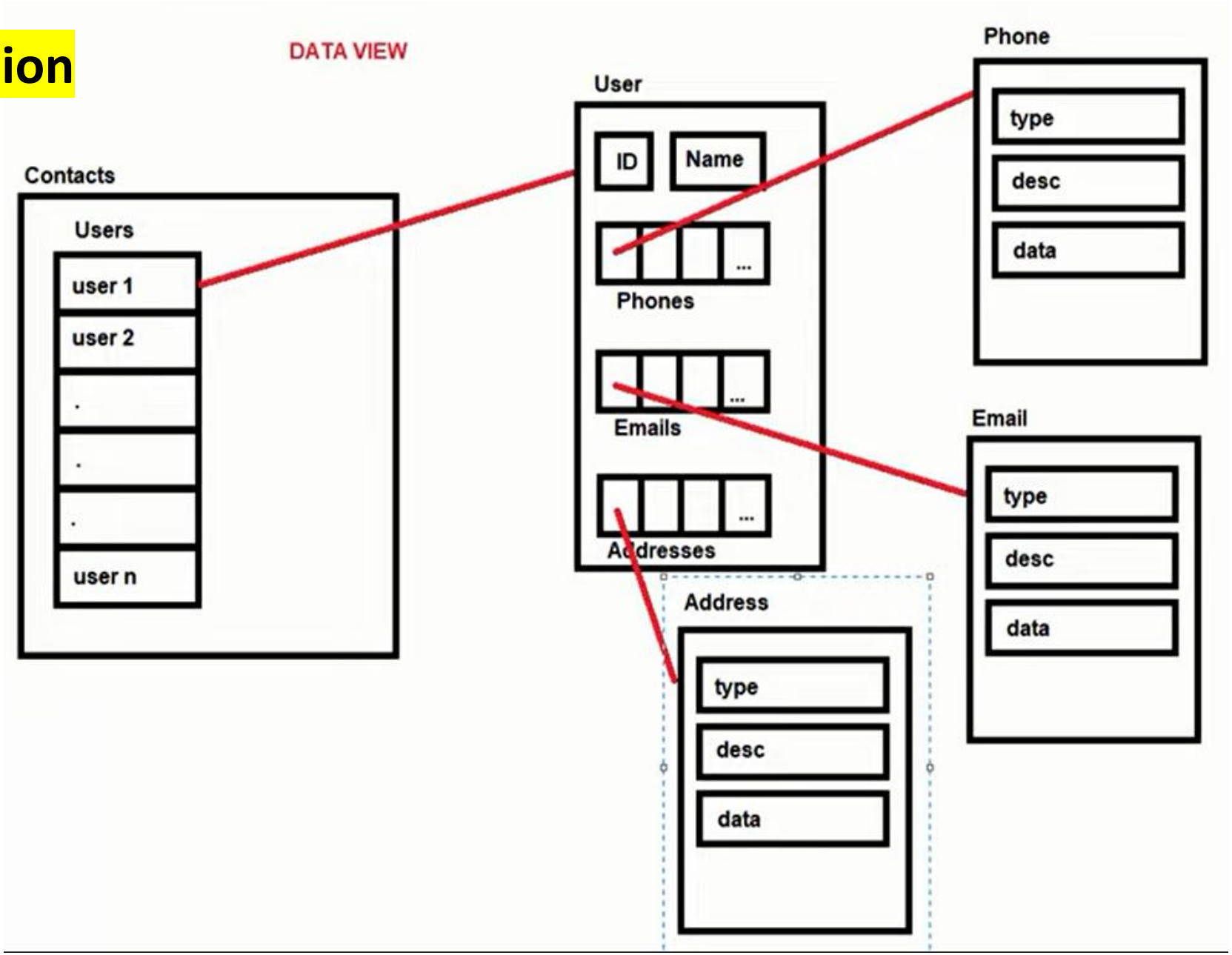
Exercise 5

```
int i, *p, *r, t[4]={0,1,2,3};
p = new int[4];
for(i=0; i<4; i++)
    p[i] = 6*(i+1);
r = t;
for(i=3; i>=0; i--)
    r[i] -= 1;
r[2] = 8;
for(i=0; i<4; i++)
    t[i] = p[i] + r[i];
for(i=0; i<4; i++)
    cout << t[i] << " ";
cout<<endl;
delete [] p;
```

Exercise 6

```
void F1(int* temp){
    *temp = 99; //will change 50 to 99
}
int main(){
    int *p1, *p2;
    p1 = new int;
    *p1 = 50;
    p2 = p1;
    F1(p2);
    cout << *p1 << " " << *p2 << endl;  // 99 99
    p1 = new int;
    *p1 = 88;
    cout << *p1 << " " << *p2 << endl;  // 88 99
    delete p1;
    delete p2;
```

Array of objects
& Dynamically allocation



Array of objects
& Dynamically allocation

```
class phone{
private:
    long phonenum;
    string type;
    string description;
public:
    phone();
};
```

```
class user{
private:
    int size;
    phone *p; // composition //pointer p point to phone type.
public:
    user(int size){
        p=new phone[size]; //DMA // dynamic array of phone' objects
    }
    ~user(){
        delete []p; //free heap from phone array
    }
};
```